

DeliveryBrief Case Study

Evidence-grounded weekly client updates from GitHub and project notes

CASE STUDY

Prepared by Elvis

7 September 2026

What I built

I built DeliveryBrief to help a project or delivery manager prepare a weekly client update from GitHub activity and project notes. It collects evidence, generates a structured draft, checks known failure conditions and leaves the external message under the manager's control.

Why I chose the title

Delivery identifies the work being reported. Brief describes the short output. I left AI out of the name because the manager needs a dependable report, not an AI interaction. I rejected DeliveryOS, ProjectPulse AI and WeeklyOps Agent because each name was less specific about the actual job.

Why I chose the workflow

Weekly delivery reporting is recurring and bounded. GitHub records system activity while project notes contain decisions, risks and client context. The workflow can be observed, its output can be compared with evidence, and a usable improvement fits within five days.

Evidence status

The software, public Streamlit app, Day 1 reconstructed weeks, direct-prompt baseline, capped Haiku evaluation and public app-run exports are complete. I still separate these from a measured time-savings claim because the app run used sample evidence.

System design

GitHub and Google Docs adapters convert source records into a shared evidence schema with stable IDs. Claude must return a typed report and attach evidence IDs to factual items. Deterministic validation then checks citations, sensitive patterns, missing action fields, empty evidence, instruction-like content and explicit source conflicts.

GitHub and Google Docs • Evidence records • Claude draft • Validation • Human approval • Exports

User control

The manager sees the source records before generation and the evidence IDs beside each draft item. Blocking findings disable approval. Warnings require review. Approval unlocks an email file, action CSV, report JSON and run record. Nothing is sent externally.

Technical choices and trade-offs

Choice	Reason	Accepted trade-off
Python and Streamlit	Small, typed and testable within five days	Less frontend control than a custom web application
Anthropic	Existing funded API account and structured output support	No cross-provider comparison in version one
Haiku and Sonnet	Measure cost against the same quality gates	One-time evaluation uses both models
No vector database	One week is a bounded evidence set	No cross-project historical retrieval
Read-only integrations	The report should not change its evidence sources	Source corrections happen outside the app
Email export	Keeps external communication under manager control	One manual step remains

Scope and reliability

Version one

- One configured project, GitHub repository and Google Drive folder
- One delivery manager workflow
- Read-only source access
- Evidence-linked report generation, review, approval and export
- Run records with model, latency, usage, findings and edits

Deliberate exclusions

- Automatic email sending
- Delivery-date prediction
- Multi-project administration and per-user OAuth
- Historical semantic search
- A general chatbot or multi-agent framework

Current engineering evidence

Eighteen automated tests pass. Ruff, mypy and the secret scan pass. The capped DeliveryBrief Haiku workflow passed 9 of 9 scored report cases, while the transient GitHub failure is covered by a mocked retry test.

Failure evidence to add

Three failures changed the system: Anthropic schema rejection, missing cross-repository evidence and disappearing action-like evidence. I fixed each issue and reran focused or full regressions before recording the final Haiku result.

Results and adoption

The direct-prompt baseline passed 1 of 9 scored cases and cost \$0.007295. The DeliveryBrief Haiku workflow passed 9 of 9 scored report cases and cost \$0.025316. The app-run evidence shows that Elvis approved and exported a sample report from the public Streamlit app. The public sample remains labeled separately from field evidence.

Main current limitation

The implementation has not yet been timed with a second target manager using their own anonymized weekly records. Until that happens, it is a working system with strong engineering evidence rather than broad adoption evidence.

Next two weeks

- Add scheduled collection after the manual pilot establishes the correct source window.
- Create Gmail drafts only after recipient controls and approval auditing are tested.
- Add per-user OAuth and a second project configuration.
- Run the workflow with three more managers and expand the case set to thirty.
- Track completion, time to approval, edit rate, warnings and abandoned runs.